Formal Verification Report: Uniswap V4 Periphery

- Repository: https://github.com/alexzoid-eth/uniswap-v4-periphery-cantina-fv
- Date: Sept 2024
- Author: @alexzoid (Cantina formal verification competition)
- Certora Prover version: latest

Table of Contents

1. About Uniswap V4 Periphery
2. Formal Verification Methodology
    - Types of Properties
    - Verification Process
    - Project Structure
    - Assumptions
3. Verification Properties
    - Valid State
    - Variable Transitions
    - State Transitions
    - High-Level
    - Unit Tests
4. Real Issues Properties
5. Mutation Testing
    - What is Mutation Testing
    - Available Mutation Sets
6. Setup and Execution
    - Common Setup (Steps 1-4)
    - Remote Execution
    - Local Execution
    - Running Verification
    - Running Mutation Testing

About Uniswap V4 Periphery

Uniswap V4 Periphery provides the user-facing contracts that interact with the Uniswap V4 core `PoolManager`. The primary contract under verification is the **PositionManager**, which manages liquidity positions as ERC-721 NFTs on top of Uniswap V4's singleton pool architecture.

The PositionManager is responsible for:

- **Minting and burning** ERC-721 position tokens tied to concentrated liquidity ranges
- **Increasing and decreasing** liquidity within existing positions
- **Settling and taking** currency balances against the PoolManager via an action-based router pattern
- **Closing and sweeping** residual balances after position operations
- **Subscribing and unsubscribing** positions to external notifier contracts for callback-based integrations
- **Nonce management** for permit-based authorization (ERC-721 Permit v4)

A single FV target was in scope: **PositionManager** (with PoolManager fully mocked as a ghost-backed dependency). The V4Router configuration is included but was not the primary verification focus.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Properties are categorized following the [official Certora methodology](). Valid State, Variable Transitions and State Transitions properties are **parametric** — they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

Valid State — System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

Variable Transitions — Properties that verify specific storage variables change only under expected conditions. The process captures a variable value before instruction execution, runs the instruction, then asserts the variable changed only as permitted or remained unchanged.

State Transitions — Properties that verify the correctness of transitions between valid states. Building upon the valid state invariants, these properties ensure the protocol's state machine operates correctly and that state changes are both authorized and sequentially valid.

High-Level — Complex multi-step properties verifying business logic integrity. Unlike parametric properties, these target specific function sequences to validate end-to-end protocol behavior.

Unit Tests — Properties that verify basic behavior of individual functions: revert conditions, direct effects on state, and non-effects on unrelated state. Unlike parametric properties, each unit test targets a specific function call.

Verification Process

1. **Setup phase**: Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish the verification harness and configuration. This phase addresses several prover limitations:
    - PoolManager mock (`PoolManagerBase.spec`): The PoolManager is a complex singleton contract with AMM math, tick crossing, and fee accounting. All 14+ external functions (`initialize`, `modifyLiquidity`, `swap`, `donate`, `sync`, `take`, `settle`, `settleFor`, `clear`, `mint`, `burn`, `updateDynamicLPFee`, `setProtocolFee`, `collectProtocolFees`) are replaced with ghost-backed CVL implementations that model essential state transitions (pool slot0, liquidity, position liquidity, currency deltas, ERC-6909 balances) without the implementation complexity that causes solver timeouts.

- Transient storage (`TransientStorage.spec`): Uniswap V4 uses EIP-1153 transient storage extensively for currency deltas, lock state, and locker identity. Since the prover does not natively model transient storage semantics, `ALL_TSTORE` and `ALL_TLOAD` global hooks intercept every transient read/write and redirect them to persistent ghost variables (`ghostCurrencyDelta`, `ghostSyncedCurrency`, `ghostSyncedReserves`, `ghostLock`, `ghostNonzeroDeltaCount`, `ghostLocker`), keyed by known transient slot constants.
- ERC-20 model (`ERC20.spec`): Real ERC-20 contracts cause timeouts due to implementation complexity and unbounded address space. A lightweight ghost-variable-based model summarizes all external ERC-20 calls (`balanceOf`, `transfer`, `transferFrom`) over a bounded set of 4 currencies (native + 3 ERC-20 tokens: `ghostERC20A`, `ghostERC20B`, `ghostERC20C`) with ordering axioms to model valid pool key currency pairs.
- Math summaries (`MathCVL.spec`): `FullMath.mulDiv` and `FullMath.mulDivUp` use intermediate multiplication that overflows in the solver. They are replaced with exact CVL expressions: `mulDiv(x,y,z) = x*y/z` and `mulDivUp(x,y,z) = (x*y + z - 1)/z`.
- StateLibrary/TransientStateLibrary redirects (`StateLibrary.spec`, `TransientStateLibrary.spec`): Library functions that read pool state via `extsload` / `exttload` are redirected to ghost variable lookups, bypassing raw storage slot calculations.
- Hooks disabled (`PoolManagerBase.spec`): `Hooks.callHook()` is summarized as a no-op stub (`callHookStubCVL`), excluding hook contract interactions from verification scope.
- Removed entry points (`PositionManagerValidState.spec`): `modifyLiquidities`, `modifyLiquiditiesWithoutUnlock`, `unlockCallback`, and `multicall` are summarized as `NONDET DELETE`, as the action-based routing is verified through individual action handlers.

2. **Crafting Properties**: Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules), then variable/state transitions, and finally high-level and unit test properties.

Project Structure

```
certora/
├── confs/                                    # Prover configuration files
│   ├── PositionManager_part1_verified.conf    # PM rules 1-11 (VT + ST)
│   ├── PositionManager_part2_verified.conf    # PM rules 12-22 (HL + UT)
│   ├── PositionManager_ValidState_verified.conf # All 16 invariants
│   ├── PositionManager_violated.conf          # Real issue demonstration
│   └── V4Router.conf                          # V4Router verification
│
├── harnesses/                                # Verification harnesses
│   ├── HelperCVL.sol                          # CVL helper: encoding/decoding, extraction
│   ├── PoolManagerHarness.sol                 # PoolManager mock harness
│   ├── PositionManagerHarness.sol             # Primary contract under test
│   ├── Receiver.sol                           # Mock ERC721/ERC20 receiver
│   └── V4RouterHarness.sol                    # V4Router harness
│
├── mutations/                                # Mutation testing files
│   ├── DeltaResolver/                         # DeltaResolver mutations (2)
│   │   ├── DeltaResolver_0.sol
│   │   └── DeltaResolver_1.sol
```

```
│   ├── PositionManager/                      # PositionManager mutations (11)
│   │   ├── PositionManager_0.sol
│   │   ├── PositionManager_1.sol
│   │   ├── PositionManager_2.sol
│   │   ├── PositionManager_3.sol
│   │   ├── PositionManager_4.sol
│   │   ├── PositionManager_5.sol
│   │   ├── PositionManager_6.sol
│   │   ├── PositionManager_7.sol
│   │   ├── PositionManager_8.sol
│   │   ├── PositionManager_P0.sol
│   │   └── PositionManager_P1.sol
│   ├── PositionManager_fix/                  # Fix mutation for real issue
│   │   └── 1.sol
│   └── V4Router/                             # V4Router mutations (5)
│       ├── V4Router_0.sol
│       ├── V4Router_1.sol
│       ├── V4Router_2.sol
│       ├── V4Router_3.sol
│       └── V4Router_P.sol
│
├── scripts/                                  # Automation scripts
│   ├── add_mutation.sh                       # Register new manual mutations
│   ├── certora_mutate_all.sh                 # Run all mutation testing
│   └── check_mutation.sh                     # Validate mutation integration
│
└── specs/                                    # CVL specifications
    ├── PositionManager.spec                  # 22 rules (VT + ST + HL + UT)
    ├── PositionManagerValidState.spec        # 16 invariants (Valid State)
    ├── PositionManagerViolated.spec          # Real issue demonstration (PMR-01)
    └── setup/                                # Shared setup: ghosts, hooks, summaries
        ├── Constants.spec                    # Protocol constants (ticks, fees)
        ├── ERC20.spec                        # Ghost-based ERC20 token model
        ├── HelperCVL.spec                    # CVL helpers (pool ID, position key)
        ├── MathCVL.spec                      # Math library summaries (mulDiv)
        ├── PoolManagerBase.spec              # PoolManager ghost-backed mock
        ├── PositionManagerBase.spec          # PM ghost variables and hooks
        ├── StateLibrary.spec                 # StateLibrary ghost redirects
        ├── TransientStateLibrary.spec        # TransientStateLibrary ghost redirects
        ├── TransientStorage.spec             # EIP-1153 transient storage hooks
        └── V4RouterBase.spec                 # V4Router setup
```

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are categorized into three groups: **Safe** (real-world constraints that don't reduce security coverage), **Proved** (formally verified invariants reused as preconditions), and **Unsafe** (scope reductions necessary for tractability that may exclude valid scenarios).

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees.

Environment Constraints (`PoolManagerBase.spec`):

- The sender is never the zero address and never the reserved addresses 1 or 2 (used by `_mapRecipient()` for `MSG_SENDER` and `ADDRESS_THIS` constants)
- The PositionManager address itself is not equal to reserved constants 1 or 2

Token Model (`ERC20.spec`):
The ERC-20 model supports exactly 4 currencies: native ETH (`address(0)`) and 3 ERC-20 tokens (`ghostERC20A < ghostERC20B < ghostERC20C`). Token addresses are constrained to be non-zero, distinct from the PositionManager and PoolManager, and strictly ordered. All balances and allowances are bounded to `max_uint128`.

Pool Key Validity (`PoolManagerBase.spec`):

- Only two pool key configurations are modeled: `NATIVE/ERC20B` or `ERC20A/ERC20B` (currency0 < currency1)
- Tick spacing is within `[1, 32767]` (valid Uniswap V4 range)
- LP fee is at most `MAX_LP_FEE` (1000000) or equals the `DYNAMIC_FEE_FLAG`

Locker Identity (`TransientStorage.spec`):

- The locker address (set via transient storage) is constrained to be distinct from both the PoolManager and the PositionManager, reflecting that the locker is always an external caller

Math Summaries (`MathCVL.spec`):

- `FullMath.mulDiv(x, y, z)` replaced with exact CVL arithmetic `x * y / z`
- `FullMath.mulDivUp(x, y, z)` replaced with `(x * y + z - 1) / z`
- `divUp(x, y)` replaced with `(x + y - 1) / y`

Proved Assumptions

These properties have been formally verified as valid state invariants and are used as trusted preconditions (via `requireInvariant`) in state transition and high-level rules. See Valid State for detailed descriptions and verification statuses.

PositionManager Token Invariants (`PositionManagerValidState.spec`):

- Next token ID is always ≥ 1; no position or owner data exists for token ID 0 or unissued IDs (PMV-01, PMV-02)
- Active position ticks are within valid TickMath range with tickLower < tickUpper (PMV-03)
- Active positions correspond to minted tokens and vice versa (PMV-04)
- Active positions correspond to valid pool keys and initialized pools in PoolManager (PMV-05, PMV-06)
- Pool keys with non-zero tick spacing correspond to initialized pools (PMV-07)
- Pool key structures contain valid currencies, tick spacing, and fees (PMV-08)

PositionManager State Invariants (`PositionManagerValidState.spec`):

- Token position ID alignment: position liquidity exists only for valid token IDs (PMV-09)
- Subscriber consistency: subscriber address matches the notifier's tracked token ID (PMV-10)
- Notifier callbacks are zeroed when no subscriber exists (PMV-11)
- Subscriber flag matches non-zero subscriber address (PMV-12)
- Subscribers exist only for minted tokens with active positions (PMV-13)

ERC-721 Invariants (`PositionManagerValidState.spec`):

- No approvals for the zero address as owner (PMV-14)
- Zero address has no token balance (PMV-15)
- Approved tokens correspond to existing token IDs (PMV-16)

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover.

PoolManager Mock (`PoolManagerBase.spec`):
The entire PoolManager is replaced with ghost-backed CVL functions. Real AMM math (tick crossing, sqrt price computation, fee accumulation) is not modeled. This means scenarios involving specific price movement patterns, tick boundary behavior, or fee rounding edge cases are excluded from verification. The mock preserves essential properties: liquidity modification updates position liquidity and generates appropriate balance deltas, swaps produce valid delta signs, and currency accounting is maintained.

Hooks Disabled (`PoolManagerBase.spec`):

- `Hooks.callHook()` is summarized as a no-op stub, so hook contract interactions (before/after callbacks, dynamic fee computation) are not verified

Removed Entry Points (`PositionManagerValidState.spec`):

- `modifyLiquidities`, `modifyLiquiditiesWithoutUnlock`, `unlockCallback`, and `multicall` are summarized as `NONDET DELETE`. The action-based routing is verified through individual action handlers, but the top-level batching and unlock mechanism are excluded.

Bounded Model (`ERC20.spec`):

- Only 4 currencies are modeled (native + 3 ERC-20 tokens), with balances bounded to `max_uint128`
- Only 2 pool key configurations are supported (`NATIVE/ERC20B` or `ERC20A/ERC20B`)

Permit2 (`PositionManagerBase.spec`):

- `permit()` is summarized as `NONDET` — permit signature validation is not verified
- `transferFrom` is redirected to the ghost-backed ERC-20 model

Prover Configuration:

- Loop unrolling is capped at 2 iterations across all configurations
- Optimistic loop and optimistic fallback are enabled

Verification Properties

Links to specific CVL spec files are provided for each property, with status indicators.

- ✅ Verified
- ❌ Violated

Valid State

Valid State properties define the fundamental invariants that must always hold true throughout the PositionManager's lifecycle. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Property	Name	Description	Status	Notes
PMV-01	`validNextTokenId`	The next token ID MUST always be greater than or equal to 1	✅	
PMV-02	`noInfoForInvalidTokenIds`	Position information and token MUST NOT exist for token ID 0 or any future token IDs	✅	
PMV-03	`validPositionTicks`	Active position ticks MUST be within the valid range defined by TickMath	✅	
PMV-04	`activePositionsMatchesToken`	Active position MUST correspond to a minted token and vice versa	✅	
PMV-05	`activePositionMatchesPoolKey`	Active position MUST correspond to valid pool key	✅	
PMV-06	`activePositionMatchesInitializedPoolInPoolManager`	Active position must correspond to an initialized pool in PoolManager	✅	
PMV-07	`poolKeyMatchesInitializedPoolInPoolManager`	Touched pool key must correspond to an initialized pool in PoolManager	✅	
PMV-08	`validPoolKeyStructure`	All poolKey fields must contain valid and consistent values	✅	
PMV-09	`tokenPositionIdAlignment`	The token ID in PositionManager always matches the position ID in PoolManager	✅	
PMV-10	`subscriberTokenIdConsistency`	The subscriber token ID always matches the token ID passed to notifier callbacks	✅	
PMV-11	`notifierCallbacksUntouchedIfNoSubscriber`	Notifier callbacks are not called if nobody is subscribed to the token	✅	
PMV-12	`subscriberAddressSetWithFlag`	A subscription flag MUST be set when a valid subscriber address is present and vice versa	✅	
PMV-13	`subscribersForExistingTokensOnly`	Subscribers MUST only be set for existing token IDs	✅	
PMV-14	`noApprovalsForZeroAddress`	Approvals for the zero address as owner are not allowed	✅	

Property	Name	Description	Status	Notes
PMV-15	`zeroAddressHasNoBalance`	The zero address MUST NOT have any token balance	✅	
PMV-16	`getApprovedForExistingTokensOnly`	Approved tokens MUST always correspond to existing token IDs	✅	

Variable Transitions

Variable Transition properties verify that specific storage variables change only under expected conditions (monotonicity, set-once semantics, or authorized-function restrictions). These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Property	Name	Description	Status	Notes
PM-01	`nextTokenIdMonotonicallyIncreasing`	The nextTokenId value always increases monotonically, incrementing by exactly 1 when a new token is minted	✅	
PM-02	`positionTicksUnchanged`	The ticks in positionInfo remain constant for a given tokenId unless the position is cleared, in which case they are reset to zero	✅	
PM-03	`positionPoolIdUnchanged`	The poolId in positionInfo remains constant for a given tokenId unless the position is cleared, in which case it is reset to zero	✅	
PM-04	`subscriberImmutability`	A subscriber for a tokenId can only be set to a non-zero address once, and can only be unset to zero address afterwards	✅	
PM-05	`noncePermanence`	Once a nonce is set (used), it remains set and cannot be cleared or reused	✅	

State Transitions

State Transition properties verify the correctness of transitions between valid states. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Property	Name	Description	Status	Notes
PM-06	`nextTokenIdSyncWithMint`	The nextTokenId is updated if and only if a new token is minted, ensuring synchronization between token creation and ID assignment	✅	
PM-07	`tokenMintBurnPositionSync`	A new token is minted if and only if a new position is created, and burned when the position is cleared, maintaining a one-to-one relationship between tokens and positions	✅	
PM-08	`liquidityChangeOnMintBurn`	Minting a new position increases the position's liquidity in the PoolManager, while burning a position decreases its liquidity	✅	
PM-09	`poolKeyImmutability`	The poolKey associated with a poolId remains constant once set, and can only be set when the tickspacing is zero, ensuring consistency of pool parameters	✅	
PM-10	`poolKeyAdditionOnlyWithNewToken`	A pool key can only be added when a new token is minted, preventing unauthorized pool creation	✅	
PM-11	`notifierCallbacksOnlyOnOwnerOrLiquidityChange`	Notifier callbacks are executed only when the token owner changes or when the position's liquidity is modified	✅	
PM-12	`initiatesTokensReceiveFromPoolManagerOnly`	The contract only initiates token receives from the pool manager, preventing unauthorized token inflows	✅	
PM-13	`neverReceiveNativeTokensFromPoolManager`	The contract never receives native tokens from the pool manager, ensuring proper asset segregation	✅	
PM-14	`pairOfTokensSettlesFromLockerOnlyAndInFullAmount`	Pair of tokens can only be settled from the locker and must be settled for the full amount owed by the contract	✅	

High-Level

High-Level properties verify multi-step business logic: access control, ownership integrity, and fund flow restrictions.

Property	Name	Description	Status	Notes
PM-15	`onlyTokenOwnerOrApprovedCanModifyLiquidity`	Only the token owner or approved addresses can modify the liquidity of a position, ensuring proper access control	✅	

Property	Name	Description	Status	Notes
PM-16	`onlyTokenOwnerOrApprovedCanTransferToken`	Only the token owner or approved addresses can transfer a token, maintaining ownership integrity	✅	
PM-17	`sweepOnlyWayToTransferTokensOutsideExceptPoolManager`	Sweep is the only way to transfer tokens outside the contract, except for interactions with the PoolManager, preventing unauthorized token outflows	✅	

Unit Tests

Unit Test properties verify basic behavior of individual functions: direct effects on state, settlement semantics, and nonce management.

Property	Name	Description	Status	Notes
PM-18	`closeAffectBalanceOfLocker`	Closing a position affects the balance of the locker, ensuring proper settlement of assets	✅	
PM-19	`sweepMustOutputCurrency`	The sweep function must successfully output the specified currency to the designated recipient	✅	
PM-20	`nonceUniqueUsage`	Any valid nonce can be used exactly once, preventing replay attacks	✅	
PM-21	`nonceSingleUse`	A nonce cannot be successfully used more than once	✅	
PM-22	`destinationCanReceiveToken`	The destination address can always receive the token during a transfer	✅	

Real Issues Properties

This section documents a vulnerability discovered during the formal verification process. The property demonstrates how the formal specification detected the issue and how the fix resolves it.

PMR-01: `settleUsesFullContractBalance` — CONTRACT_BALANCE flag settle doesn't use full balance

When settling with the `CONTRACT_BALANCE` flag (`0x8000...000`), the settle action should use the contract's entire balance of the specified currency. The violated rule demonstrates that without a fix, the `payerIsUser` flag can be set to `true` simultaneously with `CONTRACT_BALANCE`, leading to inconsistent behavior where the settle action attempts to transfer from the user instead of using the contract's balance.

Violated rule:

```
/// @notice used to signal that an action should use the contract's entire balance of a
currency
/// This value is equivalent to 1<<255, i.e. a singular 1 in the most significant bit.
definition CONTRACT_BALANCE() returns uint256 =
0x8000000000000000000000000000000000000000000000000000000000000000;

// Verifies that when settling with the CONTRACT_BALANCE flag, the action uses the
contract's entire balance
rule settleUsesFullContractBalance(env e, bytes params) {

    // Assume PositionManager is in a valid state before the action
    requireValidStatePositionManagerEnv(e);

    // Decode the input parameters for the settle action
    PoolManager.Currency currency;
    uint256 amount;
    bool payerIsUser;
    currency, amount, payerIsUser = _HelperCVL.decodeSettleParams(params);

    // Record the locker's balance before the action
    mathint lockerBalanceBefore = balanceOfCVL(e, currency, ghostLocker);

    // Execute the settle action
    handleActionSettle(e, params);

    // Record the locker's balance after the action
    mathint lockerBalanceAfter = balanceOfCVL(e, currency, ghostLocker);

    // Check the contract's balance after the settle action
    mathint posBalanceAfter = balanceOfCVL(e, currency, _PositionManager);

    // If CONTRACT_BALANCE flag was set, the contract's entire balance must be used
    assert(amount == CONTRACT_BALANCE() => posBalanceAfter == 0);

    // If CONTRACT_BALANCE flag was set, the locker's balance should remain unchanged
```

```
    assert(amount == CONTRACT_BALANCE() => lockerBalanceBefore == lockerBalanceAfter);
}
```

Fix applied (`certora/mutations/PositionManager_fix/1.sol`):

```
+import {ActionConstants} from "./libraries/ActionConstants.sol";
+
 } else if (action == Actions.SETTLE) {
     (Currency currency, uint256 amount, bool payerIsUser) =
params.decodeCurrencyUint256AndBool();
+    if (amount == ActionConstants.CONTRACT_BALANCE) {
+        require(payerIsUser == false, "Payer must not be a user when current contract's
balance is used");
+    }
     _settle(currency, _mapPayer(payerIsUser), _mapSettleAmount(amount, currency));
     return;
```

Mutation Testing

What is Mutation Testing

Mutation testing validates the strength of a formal specification by injecting small syntactic changes (mutations) into the source code and verifying that the spec catches them. Each mutation represents a potential bug — a removed assignment, a flipped operator, an off-by-one error.

The certoraMutate tool automates this process: it applies each mutant to the source, runs the prover against the specified rule, and reports whether the invariant caught the introduced defect.

Available Mutation Sets

The project includes mutation testing infrastructure for three contracts:

PositionManager (11 mutations in `certora/mutations/PositionManager/`):

- 9 numbered mutations (`PositionManager_0.sol` through `PositionManager_8.sol`)
- 2 parameterized mutations (`PositionManager_P0.sol`, `PositionManager_P1.sol`)

DeltaResolver (2 mutations in `certora/mutations/DeltaResolver/`):

- `DeltaResolver_0.sol`, `DeltaResolver_1.sol`

V4Router (5 mutations in `certora/mutations/V4Router/`):

- 4 numbered mutations (`V4Router_0.sol` through `V4Router_3.sol`)
- 1 parameterized mutation (`V4Router_P.sol`)

PositionManager Fix (1 mutation in `certora/mutations/PositionManager_fix/`):

- `1.sol` — the fix for the CONTRACT_BALANCE vulnerability (see Real Issues Properties)

Mutation testing is configured in the following conf files: `PositionManager_ValidState_verified.conf`, `PositionManager_part1_verified.conf`, and `PositionManager_part2_verified.conf`. Each conf includes both Gambit-generated and manual mutation definitions.

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1-4)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup tutorial.

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx — installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.26
solc-select use 0.8.26
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

> **Note:** If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:
>
> ```
> certoraRun certora/confs/PositionManager_ValidState_verified.conf --server production
> ```

Local Execution

Follow the full build instructions in the [CertoraProver repository](). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:   https://github.com/Z3Prover/z3/releases
#   cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

Valid State (16 invariants)

```
certoraRun certora/confs/PositionManager_ValidState_verified.conf
```

Variable Transitions + State Transitions (PM-01 to PM-11)

```
certoraRun certora/confs/PositionManager_part1_verified.conf
```

Individual rules can be run with `--rule`:

```
certoraRun certora/confs/PositionManager_part1_verified.conf --rule
nextTokenIdMonotonically Increasing
certoraRun certora/confs/PositionManager_part1_verified.conf --rule positionTicksUnchanged
certoraRun certora/confs/PositionManager_part1_verified.conf --rule positionPoolIdUnchanged
certoraRun certora/confs/PositionManager_part1_verified.conf --rule subscriberImmutability
certoraRun certora/confs/PositionManager_part1_verified.conf --rule noncePermanence
certoraRun certora/confs/PositionManager_part1_verified.conf --rule nextTokenIdSyncWithMint
certoraRun certora/confs/PositionManager_part1_verified.conf --rule
tokenMintBurnPositionSync
certoraRun certora/confs/PositionManager_part1_verified.conf --rule
liquidityChangeOnMintBurn
certoraRun certora/confs/PositionManager_part1_verified.conf --rule poolKeyImmutability
certoraRun certora/confs/PositionManager_part1_verified.conf --rule
poolKeyAdditionOnlyWithNewToken
certoraRun certora/confs/PositionManager_part1_verified.conf --rule
notifierCallbacksOnlyOnOwnerOrLiquidityChange
```

High-Level + Unit Tests (PM-12 to PM-22)

```
certoraRun certora/confs/PositionManager_part2_verified.conf
```

Individual rules:

```
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
initiatesTokensReceiveFromPoolManagerOnly
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
neverReceiveNativeTokensFromPoolManager
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
pairOfTokensSettlesFromLockerOnlyAndInFullAmount
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
onlyTokenOwnerOrApprovedCanModifyLiquidity
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
onlyTokenOwnerOrApprovedCanTransferToken
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
sweepOnlyWayToTransferTokensOutsideExceptPoolManager
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
closeAffectBalanceOfLocker
certoraRun certora/confs/PositionManager_part2_verified.conf --rule sweepMustOutputCurrency
certoraRun certora/confs/PositionManager_part2_verified.conf --rule nonceUniqueUsage
certoraRun certora/confs/PositionManager_part2_verified.conf --rule nonceSingleUse
certoraRun certora/confs/PositionManager_part2_verified.conf --rule
destinationCanReceiveToken
```

Violated Rule (Real Issue)

```
certoraRun certora/confs/PositionManager_violated.conf
```

Running Mutation Testing

Run mutation testing for PositionManager rules:

```
certoraMutate certora/confs/PositionManager_ValidState_verified.conf
certoraMutate certora/confs/PositionManager_part1_verified.conf
certoraMutate certora/confs/PositionManager_part2_verified.conf
```

Or use the provided script to run all mutation tests:

```
./certora/scripts/certora_mutate_all.sh
```

Creating Mutations for Other Rules

1. **Create the mutations directory**: `mkdir -p certora/mutations/<ruleName>`
2. **Introduce a mutation** into the source contract (e.g., comment out a line, flip a condition)
3. **Run the helper script** to snapshot the mutation:

   ```
   ./certora/scripts/add_mutation.sh <ruleName> src/PositionManager.sol
   ```

   This auto-numbers the mutation file (e.g., `PositionManager_0.sol`, `PositionManager_1.sol`, ...) and embeds the diff block. The original source is automatically restored via `git restore`.
4. **Create a conf file** (copy an existing one and add the `mutations` section pointing to your new directory)
5. **Run** `certoraMutate` with your new conf file